U24AI006

ASSIGNMENT-12

1. A tech hub has 6 different project teams (P1 to P6) that need to meet tomorrow. However, there are only **3 meeting rooms** available (R1,R2,R3). Some teams have overlapping members and therefore cannot meet at the same time.

Constraints (Incompatibility List):

- P1 conflicts with: P2,P3,P6
- P2 conflicts with: P1,P3,P4
- P3 conflicts with: P1,P2,P5
- P4 conflicts with: P2,P6
- P5 conflicts with: P3,P6
- P6 conflicts with: P1,P4,P5

Task Requirements:

Initial State: Set the domain for every project variable to {R1,R2,R3}.

1. **Constraint Enforcement:** Apply AC-3 to this constraint graph.
2. **Step-by-Step Trace:** Provide a trace of the first 5 arc-reductions (e.g., "Arc (P1,P2) checked, no change").
3. **Consistency Check:** After running AC-3, determine if the problem is **arc-consistent**.

If we preemptively assign **Team 1** to **Room 1** , does AC-3 detect a failure, or does it leave a valid set of domains for the remaining rooms?

3. You are provided with a Sudoku puzzle classified as "Hard." Standard arc consistency often fails to solve such puzzles entirely but can significantly "thin out" the possibilities. Your task is to implement the AC-3 algorithm to see how much of the search space can be pruned before a human (or a backtracking algorithm) would need to start guessing.

Input Data: The following 81-character string representing the grid (0 denotes an empty cell):

```
0 0 0 0 0 6 0 0 0

0 5 9 0 0 0 0 0 8

2 0 0 0 0 8 0 0 0

0 4 5 0 0 0 0 0 0
```

```
0 0 3 0 0 0 0 0 0

0 0 6 0 0 3 0 5 0

0 0 0 0 0 7 0 0 0

0 0 0 0 0 0 0 0 0

0 0 0 0 5 0 0 0 2
```

Task Requirements:

1. **Arc Generation:** Automatically generate all 810+ binary constraints (arcs) based on Sudoku rules.
2. **State Tracking:** Output the number of values removed from all domains combined after the AC-3 algorithm terminates.
3. **Visualization:** Print the grid showing the remaining domain size for each cell (e.g., a "1" if solved, or "3" if three possibilities remain).

Key Question: Does AC-3 reduce any domain to zero (proving the puzzle unsolvable) or reduce all domains to one (solving it completely)?

CODE(1):

```
graph = {
    "P1": ["P2", "P3", "P6"],
    "P2": ["P1", "P3", "P4"],
    "P3": ["P1", "P2", "P5"],
    "P4": ["P2", "P6"],
    "P5": ["P3", "P6"],
    "P6": ["P1", "P4", "P5"]
}

rooms = ["R1", "R2", "R3"]


def make_domains():
    domains = {}
    for team in graph:
        domains[team] = rooms[:]
    return domains


def revise(domains, xi, xj):
    removed = []
    current_values = domains[xi][:]
```

```
    for value in current_values:
        supported = False
        for other in domains[xj]:
            if value != other:
                supported = True
                break
        if not supported:
            domains[xi].remove(value)
            removed.append(value)

    return len(removed) > 0, removed


def ac3(domains):
    queue = []
    for xi in graph:
        for xj in graph[xi]:
            queue.append((xi, xj))

    trace = []

    while queue:
        xi, xj = queue.pop(0)
        changed, removed = revise(domains, xi, xj)

        if len(trace) < 5:
            if removed:
                trace.append("Arc (" + xi + ", " + xj + ") checked, removed " + ",
".join(removed) + " from " + xi)
            else:
                trace.append("Arc (" + xi + ", " + xj + ") checked, no change")

        if len(domains[xi]) == 0:
            return False, trace, domains

        if changed:
            for xk in graph[xi]:
                if xk != xj:
                    queue.append((xk, xi))

    return True, trace, domains


def is_arc_consistent(domains):
```

```
    for xi in graph:
        for xj in graph[xi]:
            for value in domains[xi]:
                found = False
                for other in domains[xj]:
                    if value != other:
                        found = True
                        break
                if not found:
                    return False
    return True


def show_domains(domains):
    for team in domains:
        print(team, ":", "{" + ", ".join(domains[team]) + "}")


def is_safe(team, room, assignment):
    for neighbor in graph[team]:
        if neighbor in assignment and assignment[neighbor] == room:
            return False
    return True


def solve(order, domains, assignment, index):
    if index == len(order):
        return True

    team = order[index]

    for room in domains[team]:
        if is_safe(team, room, assignment):
            assignment[team] = room
            if solve(order, domains, assignment, index + 1):
                return True
            assignment.pop(team)

    return False


def main():
    initial_domains = make_domains()
    ok1, trace1, final1 = ac3(initial_domains)
```

```
print("Initial domains")
show_domains(make_domains())
print()

print("First 5 arc checks")
for step in trace1:
    print(step)
print()

print("Domains after AC-3")
show_domains(final1)
print()

if ok1 and is_arc_consistent(final1):
    print("The problem is arc-consistent.")
else:
    print("The problem is not arc-consistent.")
print()

assigned_domains = make_domains()
assigned_domains["P1"] = ["R1"]
ok2, trace2, final2 = ac3(assigned_domains)

print("After assigning P1 to R1")
print("First 5 arc checks")
for step in trace2:
    print(step)
print()

print("Domains after AC-3")
show_domains(final2)
print()

if not ok2:
    print("AC-3 detects failure after assigning P1 to R1.")
else:
    print("AC-3 does not detect failure after assigning P1 to R1.")
    print("A valid set of domains remains for the remaining teams.")
    print()

    order = sorted(graph, key=lambda team: len(final2[team]))
    assignment = {}
    if solve(order, final2, assignment, 0):
        print("One valid room assignment is")
        for team in graph:
```

```
                print(team, "->", assignment[team])



if __name__ == "__main__":
    main()
```

OUTPUT:

```
Initial domains
P1 : {R1, R2, R3}
P2 : {R1, R2, R3}
P3 : {R1, R2, R3}
P4 : {R1, R2, R3}
P5 : {R1, R2, R3}
P6 : {R1, R2, R3}

First 5 arc checks
Arc (P1, P2) checked, no change
Arc (P1, P3) checked, no change
Arc (P1, P6) checked, no change
Arc (P2, P1) checked, no change
Arc (P2, P3) checked, no change

Domains after AC-3
P1 : {R1, R2, R3}
P2 : {R1, R2, R3}
P3 : {R1, R2, R3}
P4 : {R1, R2, R3}
P5 : {R1, R2, R3}
P6 : {R1, R2, R3}

The problem is arc-consistent.

After assigning P1 to R1
First 5 arc checks
Arc (P1, P2) checked, no change
Arc (P1, P3) checked, no change
Arc (P1, P6) checked, no change
Arc (P2, P1) checked, removed R1 from P2
Arc (P2, P3) checked, no change

Domains after AC-3
P1 : {R1}
P2 : {R2, R3}
P3 : {R2, R3}
P4 : {R1, R2, R3}
P5 : {R1, R2, R3}
P6 : {R2, R3}

AC-3 does not detect failure after assigning P1 to R1.
A valid set of domains remains for the remaining teams.

One valid room assignment is
P1 -> R1
P2 -> R2
P3 -> R3
P4 -> R1
P5 -> R1
P6 -> R2
```

U24AI006

CODE(2):

```
puzzle = (
    "000006000"
    "059000008"
    "200008000"
    "045000000"
    "003000000"
    "006003050"
    "000007000"
    "000000000"
    "000050002"
)


def build_peers():
    peers = {}

    for row in range(9):
        for col in range(9):
            cell = row * 9 + col
            peer_set = set()

            for c in range(9):
                if c != col:
                    peer_set.add(row * 9 + c)

            for r in range(9):
                if r != row:
                    peer_set.add(r * 9 + col)

            start_row = (row // 3) * 3
            start_col = (col // 3) * 3
            for r in range(start_row, start_row + 3):
                for c in range(start_col, start_col + 3):
                    other = r * 9 + c
                    if other != cell:
                        peer_set.add(other)

            peers[cell] = sorted(peer_set)

    return peers


def build_domains():
    domains = {}
```

```
    for i in range(81):
        value = puzzle[i]
        if value == "0":
            domains[i] = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]
        else:
            domains[i] = [value]

    return domains


def build_arcs(peers):
    arcs = []
    for cell in peers:
        for peer in peers[cell]:
            arcs.append((cell, peer))
    return arcs


def count_unique_constraints(peers):
    pairs = set()
    for cell in peers:
        for peer in peers[cell]:
            if cell < peer:
                pairs.add((cell, peer))
    return len(pairs)


def revise(domains, xi, xj):
    removed = 0
    new_domain = []

    for val in domains[xi]:
        supported = False
        for other in domains[xj]:
            if val != other:
                supported = True
                break
        if supported:
            new_domain.append(val)
        else:
            removed += 1

    domains[xi] = new_domain
    return removed
```

```python
def ac3(domains, peers):
    queue = build_arcs(peers)
    total_removed = 0

    while queue:
        xi, xj = queue.pop(0)
        removed = revise(domains, xi, xj)

        if removed > 0:
            total_removed += removed

            if len(domains[xi]) == 0:
                return False, total_removed

            for xk in peers[xi]:
                if xk != xj:
                    queue.append((xk, xi))

    return True, total_removed


def print_domain_sizes(domains):
    for row in range(9):
        values = []
        for col in range(9):
            cell = row * 9 + col
            values.append(str(len(domains[cell])))
        print(" ".join(values))


def main():
    peers = build_peers()
    domains = build_domains()
    arcs = build_arcs(peers)

    success, total_removed = ac3(domains, peers)

    print("Sudoku AC-3")
    print("Directed arcs generated:", len(arcs))
    print("Unique binary constraints:", count_unique_constraints(peers))
    print("Values removed by AC-3:", total_removed)
    print()
    print("Remaining domain size grid")
```

```python
    print_domain_sizes(domains)
    print()

    empty_found = False
    solved_all = True

    for cell in domains:
        size = len(domains[cell])
        if size == 0:
            empty_found = True
        if size != 1:
            solved_all = False

    if not success or empty_found:
        print("AC-3 reduced at least one domain to 0.")
        print("The puzzle would be inconsistent with these constraints.")
    elif solved_all:
        print("AC-3 reduced all domains to 1.")
        print("The puzzle is solved completely.")
    else:
        print("AC-3 did not reduce any domain to 0.")
        print("AC-3 also did not solve the puzzle completely.")


if __name__ == "__main__":
    main()
```

OUTPUT:

```
Sudoku AC-3
Directed arcs generated: 1620
Unique binary constraints: 810
Values removed by AC-3: 214

Remaining domain size grid
5 4 4 7 6 1 7 6 6
5 1 1 5 5 3 6 6 1
1 4 3 6 5 1 7 6 7
4 1 1 6 6 3 7 7 5
4 5 1 8 7 5 7 7 5
4 5 1 6 6 1 6 1 4
7 6 4 7 7 1 7 6 6
8 7 5 7 7 4 8 7 7
7 6 4 6 1 3 7 7 1

AC-3 did not reduce any domain to 0.
AC-3 also did not solve the puzzle completely.
```